

Activation Sparsification in Transformers: Pressure, Thresholding, and Site Placement

Abstract

Activation sparsity can reduce inference cost, but the value of a sparsification intervention depends on where it acts and what accompanies it. We study pressure toward zero, thresholding and site placement through 54 matched pretraining conditions on MiniPile, using Pythia-14M, 70M and 410M. We measure validation loss, activation distributions and model-wide sparsity: the fraction of counted scalar multiplications with a zero activation operand. Pressure’s added value changes with threshold and target sites. A seven-site recipe can yield greater model-wide sparsity than a four-site recipe despite fewer feed-forward (FFN) zeros, because it also sparsifies internal attention products. At the largest tested threshold, the seven-site recipe with pressure improves both loss and sparsity over its four-site counterpart at all three sizes. On one RTX5090, a selected fused implementation achieves a $1.234\times$ geometric mean full-model speedup across the 30 14M checkpoints for uncached, 2,048-token inference. Sparse paths add only $1.043\times$ over the same fusion on average, and attention skipping slows every checkpoint. By connecting matched intervention effects to activation distributions, affected computation and measured execution, the study provides a basis for choosing where and how to induce useful sparsity.

1 Introduction

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computation and memory access associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise a practical design question: how should a researcher place and combine sparsification interventions to obtain more useful sparsity at an acceptable quality cost? Answering this question matters when choosing which components of a recipe to retain, where to invest additional training, and which operations justify sparse-kernel development.

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc thresholding to pretrained models, ProSparse uses L1 regularization on FFN activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup and how their effects interact when combined. An intervention that works well in isolation may add little on top of another or incur a different quality cost when applied at additional sites, making sparsity alone insufficient to identify which choices drive the improvement.

We study three choices: *pressure*, which encourages selected activations toward zero; *thresholding*, which sets values to exactly zero; and *site placement*, which specifies where each acts. We ask how much pressure adds at a given threshold, and how that response changes when thresholding and pressure target more sites. We compare these components under matched pretraining conditions. This component study tests their additional and combined effects under a common setup; it does not reproduce each prior method’s full training and allocation procedure.

Zero fractions alone do not describe the affected computation. Shrinking activations need not create exact zeros, and zeros at one site may affect only a small share of the forward pass. Applying thresholding at more sites can expose more computation to sparsification, but also changes what the model must learn. We therefore measure both how the activation distributions change and which operations their zeros affect.

Our *model-wide sparsity* measure, $\mathcal{S}_{\text{model}}$, counts the fraction of forward matrix-multiplication products with a zero activation operand. An architecture-dependent sparsity ceiling, $\mathcal{S}_{\text{model}}^{\text{max}}$, gives the fraction reached if the selected sites had 100% activation sparsity. Together, they distinguish learned

sparsity from the amount of computation the intervention can affect. Direct timing then tests whether kernels can exploit those zeros on hardware.

We pretrain 54 Pythia-family models from random initialization: 30 conditions at 14M parameters and 12 each at 70M and 410M (Biderman et al., 2023). A sparsification intervention ladder organizes local FFN pressure, four-site and seven-site nonlinearities with or without pressure. Within each size, models share initialization, data order and token budget. The 14M comparisons isolate individual additions; the larger models test whether selected recipe relationships persist.

Pressure’s additional value depends on the threshold and target set. The seven-site recipe with pressure can produce greater model-wide sparsity despite fewer FFN zeros, because it also sparsifies attention products. At the largest tested threshold, it improves both loss and sparsity over its four-site counterpart at all three sizes. A specialized inference implementation achieves a $1.234\times$ geometric mean speedup across the 30 Pythia-14M checkpoints on one RTX5090, although attention skipping adds overhead. These results connect intervention choices to learned zeros, affected computation and measured execution cost. The contribution is a matched account of when pressure adds useful sparsity, why its location changes the model-wide result, and which execution costs determine the benefit on hardware.

2 Related Work

2.1 Sparsification interventions

Post-hoc thresholding removes activations that a trained model can tolerate losing. CATS thresholds small-magnitude activations within FFNs, with both training-free and fine-tuning settings (Lee et al., 2024). TEAL extends magnitude thresholding to attention and FFN projections and uses block-wise greedy allocation to account for their different sensitivities (Liu et al., 2024). These studies show that site placement and sparsity allocation affect quality even without retraining. Our uniform clipping control holds allocation fixed to provide a simple reference for the trained interventions.

Training-time interventions allow the representation to adapt to sparse computation. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023); ProSparse combines ReLU, progressive ℓ_1 pressure, and threshold shifting during continued training (Song et al., 2024). ProSparse’s schedule ablations show that comparable FFN sparsity can carry different quality costs. Sparsing Law studies how sparsity changes with training data, activation function and model architecture (Luo et al., 2025), while Cetin et al. (2026) demonstrate highly sparse FFNs induced by ℓ_1 regularization during pretraining. Together, these studies show that training can reshape the quality–sparsity trade-off. We examine how pressure’s additional effect changes with the threshold and target sites, and weight the resulting zeros by the computation they affect.

Extending sparsification beyond FFNs makes the choice of sites explicit. Q-Sparse applies top- k selection to attention and FFN projection inputs, reports overall sparsity and activated-parameter budgets, and ablates both the sparsifying nonlinearity and the straight-through estimator (Wang et al., 2024). Spark Transformer combines sparse FFNs with sparse attention through token-position selection and examines the role of its low-cost predictor (You et al., 2025). Projection-input sparsity and attention-position sparsity affect different work. Our seven-site intervention also thresholds feature coordinates in the query, key and value tensors used inside attention. We compare these broader nonlinearities with projection-input nonlinearities, both with and without pressure at 14M.

2.2 Inference speedup

Converting activation sparsity into speedup requires an execution strategy suited to the workload. TEAL accelerates batch-one autoregressive decoding, where skipping weight accesses is valuable, and reports up to $1.8\times$ speedup at 50% sparsity under its projection-based accounting (Liu et al.,

2024). Cetin et al. (2026) address batched matrix multiplication: their TwELL format packs sparse activations as a projection produces them and supports fused FFN computation, reducing the overhead of materializing and consuming the sparse representation. They evaluate throughput for full-sequence forward passes, a different workload from token-by-token decoding. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory traffic must repay indexing, packing and launch overheads; operations that remain dense limit the full-model gain.

The execution strategy also depends on shape and sparsity structure. With larger batches, different token masks can reduce the weight-loading savings available from sparsity, as Spark’s batching analysis illustrates (You et al., 2025). Block Q-Sparse introduces structure into the mask to accommodate batched execution (Wang et al., 2024); TwELL instead makes packing and fusion compatible with tiled computation (Cetin et al., 2026). Iterative kernel generation with execution and profiling feedback offers a tool for exploring these adaptations (Ouyang et al., 2025; Luo et al., 2026). We use this approach in a human-guided kernel case study, requiring numerical agreement and full-model timings that include activation-dependent selection and conversion costs.

3 Methodology

3.1 Sparsification interventions

Q-Sparse selects the top- k activation magnitudes (Wang et al., 2024). Spark Transformer motivates its statistical top- k approximation by the overhead of sorting-based selection (You et al., 2025). We take a simpler route: extend the FFN design of elementwise nonlinearities to the query, key and value operands of self-attention, using a symmetric threshold. A fixed cutoff needs neither ranking nor input-dependent moment estimates. It leaves the retained activation count free to change as training reshapes the distribution, instead of targeting a prescribed count.

A sparsification intervention specifies where to set activations to zero (nonlinearity sites $\mathcal{N}$), the nonlinearity and threshold, and where to encourage small activations during training (pressure sites $\mathcal{P}$). The two site sets need not coincide.

We use ReLU and two thresholding nonlinearities: $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. One-sided thresholding removes negative values and small positive values; symmetric thresholding removes small magnitudes while preserving the signs and values of survivors. At $\kappa = 0$, their forward maps are ReLU and the identity, respectively. Appendix A specifies boundary and derivative conventions.

The threshold is fixed within each training condition. Our hypothesis is that the learned distribution can adapt to this nonlinearity, with activation pressure providing an additional way to shape it. We examine that adaptation through the quality–sparsity comparisons and signed activation distributions.

Pressure uses $\mathcal{L}_1$: we take the mean absolute activation in each targeted site–layer tensor, after the nonlinearity where one is applied, then average these means equally. *Naive L1* optimizes $\mathcal{L}_{\text{task}} + \lambda\mathcal{L}_1$. *Orthogonal L1* (OL1) adapts gradient-conflict projection (Yu et al., 2020; Hsieh et al., 2024) to this pressure objective. It makes a task-only AdamW update, then adds a pressure correction preconditioned by the task’s second-moment estimates. When this correction opposes the adaptive task direction, OL1 projects out its conflicting component. A positive trust budget caps its norm relative to that task direction. This limits the pressure correction without guaranteeing preserved task loss; Appendix A.2 defines the procedure.

3.2 Sparsity and architectural reach

Activation sparsity is the fraction of exactly zero entries at a named site. To weight zeros by the work they affect, we define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, reported as $100\mathcal{S}_{\text{model}}\%$. Here, N_0 counts scalar multiplications with at least one zero activation operand in the declared transformer-block matrix products. N_{model} counts all valid multiplications in those operations and the dense language-model head; the head receives no zero credit. Counts are pooled before division. The percentage refers to this declared multiplication workload, not all operations or elapsed time. We count only causally valid pairs in a full, uncached sequence; Appendix B gives the exact counters.

The *model-wide sparsity ceiling*, $\mathcal{S}_{\text{model}}^{\max} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, counts the fraction of that workload whose zero operand is guaranteed by setting the selected sites to 100% activation sparsity, crediting each affected multiplication once. It depends on the architecture, site set, and sequence length, but not pressure or a checkpoint. The ceiling characterizes selected-site reach rather than sparsity achievable at acceptable quality: observed $\mathcal{S}_{\text{model}}$ also includes natural zeros outside that reach. Appendix B.2 defines the reach accounting.

4 Experimental study

We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023) on MiniPile (Kaddour, 2023). Detailed comparisons use 14M; the larger models cover selected recipes. Quality is next-token validation cross-entropy, where lower is better. We measure loss and $\mathcal{S}_{\text{model}}$ in the same forward pass over all 338 complete 2,048-token blocks from the 500 MiniPile validation documents, excluding the 1,444-token tail. All training-result figures and tables use this evaluation.

Table 1 defines the recipes. The sites are the normalized attention and FFN inputs (a, m), FFN hidden activation (h), concatenated attention context (z), and query, key and value operands (q, k, v). Query and key nonlinearities follow rotary position encoding (RoPE); z precedes the attention-output projection. Each recipe is pretrained separately. Appendix C.1 maps these sites onto the shared Pythia block.

Table 1: Sparsification recipes. Selected nonlinearities replace GELU at h and act on the existing activation at other sites. OL1 targets every selected site in its row. All recipes are evaluated at 14M; 70M and 410M use A0, A1-H, A4-OL1 and A7-OL1.

Recipe	Nonlinearities	Pressure
A0	Stock GELU at h	None
A1-H	ReLU at h	None
A1-H-L1 / -OL1	ReLU at h	L1 / OL1 at h
A4 / A4-OL1	$G_{+, \kappa}$ at a, m, h, z	None / OL1 at four sites
A7 / A7-OL1	A4 plus $G_{\pm, \kappa}$ at q, k, v	None / OL1 at seven sites

Within each size, matched contrasts share initialization, data order, and training budget. Adding pressure at fixed nonlinearities measures its additional quality and sparsity effects; repeating the contrast across thresholds tests how those effects depend on thresholding. Comparing A4 with A7 without pressure examines broader nonlinearity placement. A4-OL1 versus A7-OL1 changes both nonlinearity and pressure sites, so it compares complete recipes rather than isolating nonlinearity placement at a fixed pressure objective.

The cohort contains 30 conditions at 14M and 12 at each larger size. Each trains for 712 updates and 1.493 billion input tokens; we evaluate the final checkpoint. Local pressure varies $\lambda \in \{0.05, 0.1, 0.5, 1\}$; A4/A7 vary $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, and all OL1 conditions use trust budget 1. Sweeping these settings tests whether an added intervention has a consistent effect or depends on the strength of the existing intervention. Appendix C.4 gives the optimizer, precision and learning-rate settings.

For the evaluation-only reference, we apply uniform magnitude-clipping targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z without updating weights. Site-layer thresholds are calibrated separately on ten training blocks. This is a uniform TEAL-style control (Liu et al., 2024), not a reproduction of an optimized allocation across sites. The main figures show A0 clipping; Appendix D.3 includes all 540 evaluations across the 54 trained checkpoints.

4.1 Quality–sparsity trade-offs

Local pressure and broader nonlinearities offer different quality–sparsity trade-offs (Figure 1). With OL1 at h , the model can improve validation loss while introducing modest model-wide sparsity: at $\lambda = 0.5$, A1-H-OL1 reaches 3.768% sparsity at loss 5.1102, compared with A0’s loss of 5.2086. A0 post-hoc clipping reaches a similar sparsity level, 3.832% at target $p = 0.3$, but with loss 5.3619. Thus, similar aggregate sparsity can carry different quality costs depending on the trained recipe and evaluation-time intervention.

Broader interventions reach much greater sparsity. A7-OL1 at $\kappa = 0.5$ attains 27.483% model-wide sparsity at loss 5.8294, accepting a quality cost relative to A0. This comparison also changes computational reach: the A0 clipping reference acts at a, m, h, z , whereas A7 additionally thresholds the internal attention operands. At 14M, these site sets have analytic sparsity ceilings of 12.833% and 29.952%, respectively. This comparison changes both the trained representation and the computation the selected sites can affect. Paired comparisons separate the additions that produce the trade-off. Full endpoints and clipping trajectories are given in Appendix D.

4.2 Paired intervention effects

Pressure’s additional value depends on the threshold and target set (Figure 2). Replacing GELU with ReLU alone increases model-wide sparsity by 2.714 percentage points (pp), but raises loss by 0.0611. Adding local pressure improves loss relative to this ReLU control. Changing naive L1 to OL1 lowers loss at three of the four matched weights, but reverses that improvement at $\lambda = 1$; the update rule is not uniformly better across the sweep.

For A4, adding OL1 improves both loss and sparsity at $\kappa = 0$ and 0.01. At larger thresholds it adds sparsity at increasing quality cost. At $\kappa = 0.5$, the increment is 2.498 pp for +0.3783 loss. The corresponding A7 pressure addition yields 12.096 pp for +0.1265 loss, while at $\kappa = 0.1$ it yields 1.372 pp for a much smaller +0.0008 loss change. Broader thresholding also contributes without pressure: A7 minus A4 at $\kappa = 0.5$ adds 5.171 pp for +0.0432 loss.

Pressure therefore needs to be evaluated at its actual threshold and sites. Its larger gain under A7 also involves more pressure targets and a different objective normalization; it cannot be assigned to nonlinearity placement alone. We next examine the distributions learned by these complete recipes.

4.3 Reshaping activation distributions

More FFN zeros need not mean more model-wide sparsity (Figure 3). At $\kappa = 0.5$, A4-OL1 places 99.31% of pooled FFN elements exactly at zero, compared with 93.64% for A7-OL1. Their attention distributions differ more sharply: A4-OL1 has a narrow nonzero peak and only 0.22% exact zeros, whereas A7-OL1 has 95.60% exact zeros and nonzero density outside the symmetric threshold gap. The omitted zero masses are reported in Table 8; a narrow density peak is not itself a large exact-zero probability.

A4 does not directly threshold or pressure q, k, v , yet their distributions change substantially. This is a nonlocal response under the complete A4-OL1 intervention, consistent with spillover. The comparison with A0 does not isolate pressure’s contribution from the other training changes. A7 directly intervenes at these attention sites, so its reshaping there has a different interpretation.

Quality vs. model-wide sparsity frontier for
train-time and post-hoc interventions (Pythia-14M)

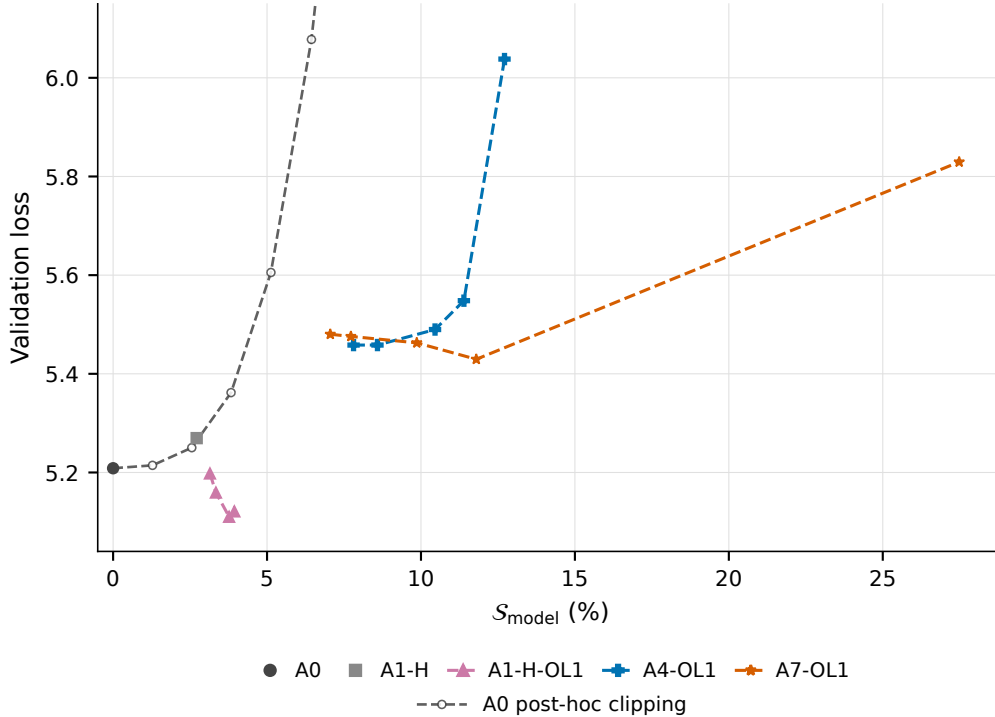


Figure 1: Quality-sparsity trade-offs from training interventions and post-hoc clipping. Validation loss versus model-wide sparsity S_{model} for Pythia-14M. Filled markers show 16 separately trained checkpoints: A0, A1-H, and the A1-H-OL1, A4-OL1 and A7-OL1 sweeps. The gray dashed path with open markers applies evaluation-only magnitude clipping at a, m, h, z to A0, with calibration targets $p = 0, 0.1, \dots, 0.9$. Six of its ten evaluations lie inside the displayed loss range; Appendix D.3 supplies the full-range view. Lines connect evaluated settings, not fitted Pareto envelopes. Local pressure reaches lower loss with modest sparsity; A7-OL1 reaches greater sparsity at a quality cost and with broader computational reach. All points use the matched protocol in Section 4.

The operation counts show why this distinction matters model-wide. At the same threshold, QK and PV contribute 16.77 sparsity points under A7-OL1, versus 0.06 under A4-OL1 (Appendix D.2). Several projection contributions are smaller under A7-OL1, but its attention contribution more than offsets them. Greater model-wide sparsity can therefore coexist with less pooled FFN sparsity. The scale comparison below tests whether the A7-OL1 advantage over A4-OL1 persists in larger models.

4.4 Transfer across model sizes

The high-threshold A7-OL1 advantage persists at all three model sizes (Figure 4). At $\kappa = 0.5$, A7-OL1 has both lower loss and greater model-wide sparsity than A4-OL1 at 14M, 70M and 410M (Table 2). The zero-threshold ordering does not transfer unchanged: A4-OL1 is better on both axes at 14M and 70M, whereas A7-OL1 is better at 410M. Persistence concerns a specific recipe relationship, not the entire threshold response.

Raw sparsity must also be read against the changing workload. The A7 sparsity ceiling rises from 29.95% to 49.42% and 87.25% as the dense LM head accounts for a smaller share of counted

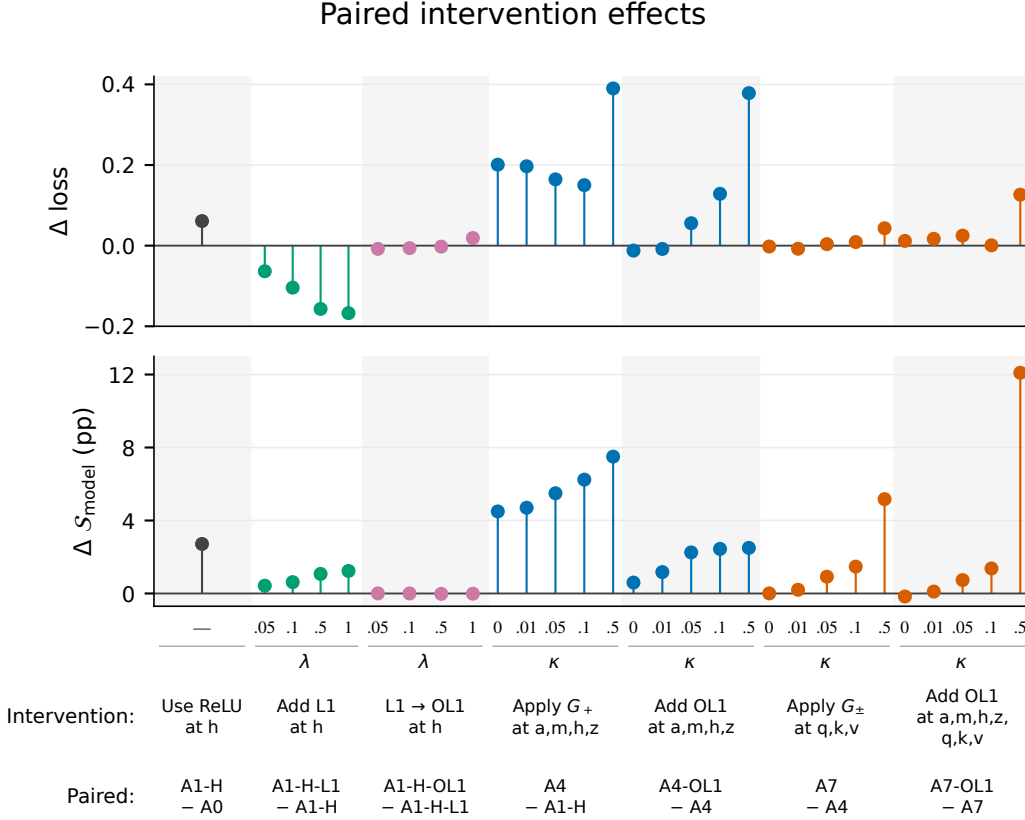


Figure 2: Pressure’s added value depends on the threshold and target sites. Changes in validation loss (top; negative is better) and $100\mathcal{S}_{\text{model}}$ (bottom; percentage points) for 29 matched Pythia-14M comparisons. Each delta is treatment minus reference, as specified in the **Paired:** row; the **Intervention:** row gives the operation and sites. Labels give λ or κ in increasing order with ordinal spacing; the dash denotes the unswept ReLU replacement. L1-to-OL1 comparisons share λ ; A4/A7 pressure additions and the Q/K/V thresholding addition share κ . Other comparisons use the fixed A0 or A1-H control. Pairs share initialization, data order and budget. These are trained endpoint differences, not successive training updates.

products. The bottom row uses the same A7 denominator for every curve within a size: $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$. Because A7 reaches all counted block operations, this ratio equals the block zero-product fraction $\mathcal{S}_{\text{block}}$. It removes the dense-head share while retaining the different weights of operations within the blocks.

This is transfer of a complete-recipe comparison. The larger cohorts lack the no-pressure A4/A7 controls needed to isolate pressure’s contribution. They share a token budget rather than tokens per parameter, and 410M uses a lower peak learning rate. Its A0 loss is also higher than 70M’s under this budget; the plot should not be read as a quality scaling law.

How interventions reshape activation distributions (Pythia-14M)

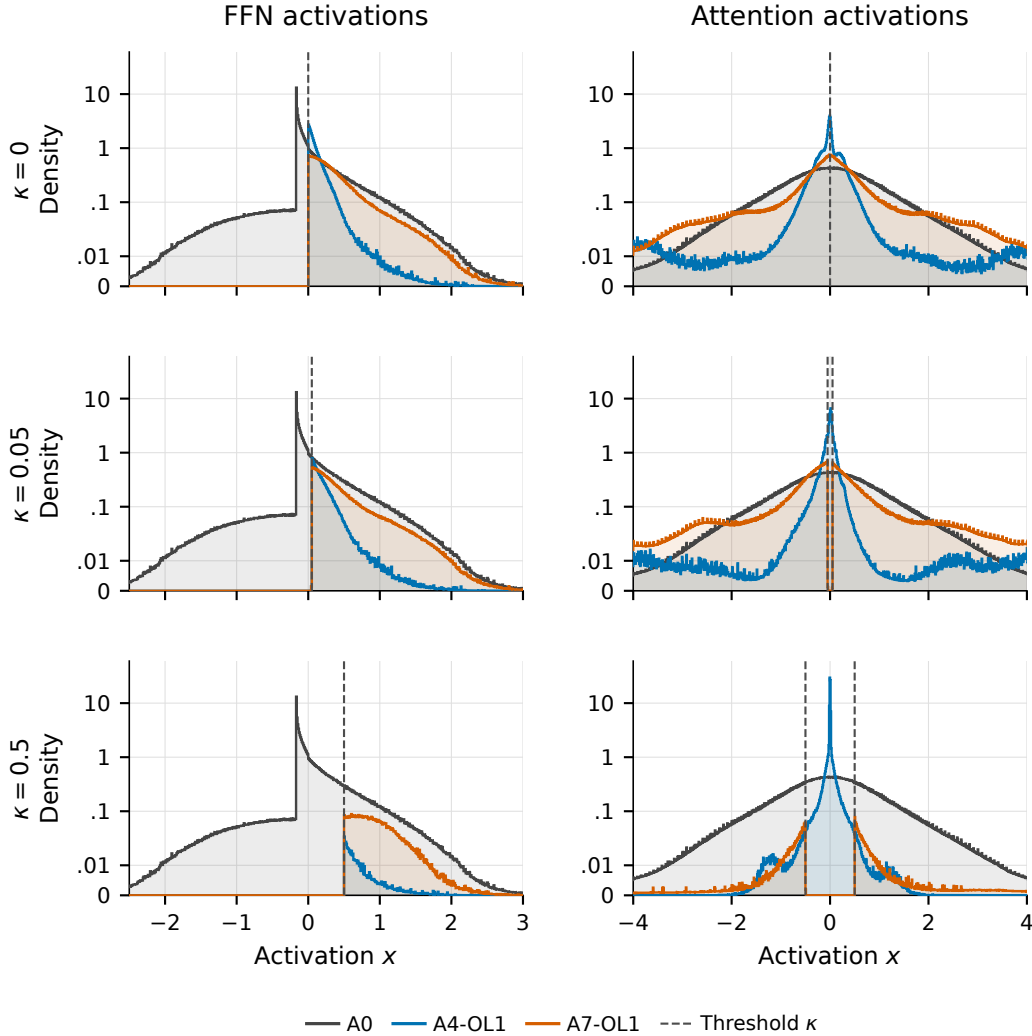


Figure 3: Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 8. Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc clipping.

Quality vs. model-wide sparsity frontier
across model sizes (Pythia-family)

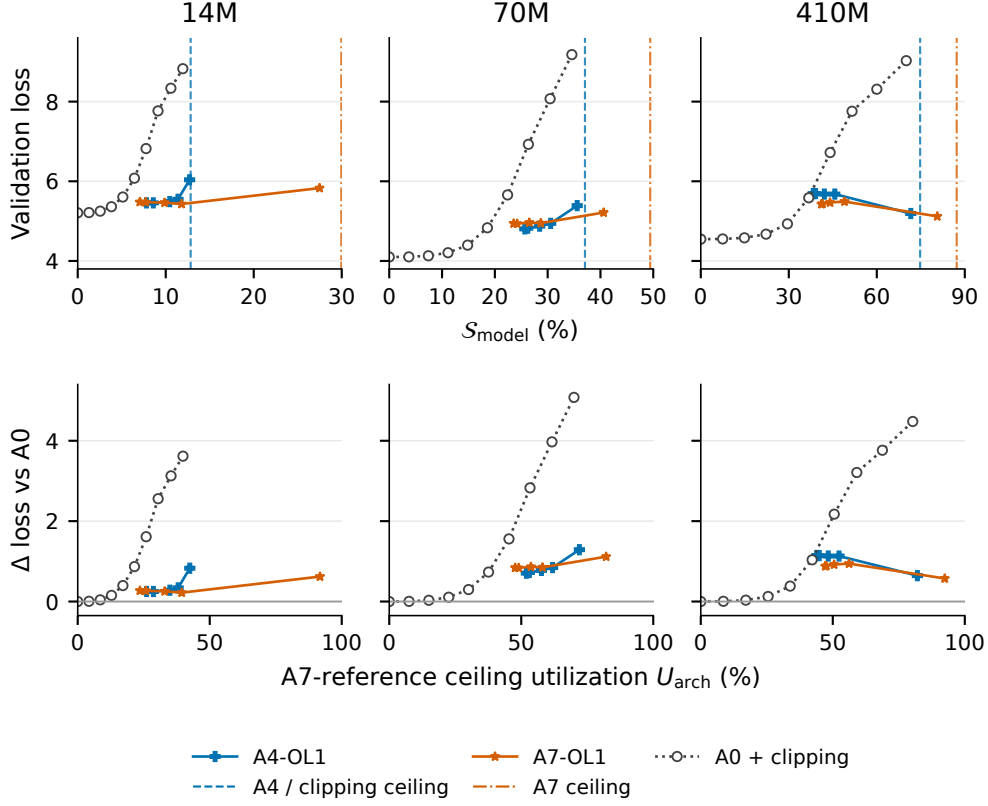


Figure 4: The high-threshold A7-OL1 advantage persists across model sizes. Columns compare 14M, 70M and 410M. Solid curves connect separately trained A4-OL1 and A7-OL1 checkpoints at $\kappa = 0, 0.01, 0.05, 0.1, 0.5$; dotted open-marker curves show A0 post-hoc clipping at a, m, h, z for targets $p = 0, 0.1, \dots, 0.9$. The 60 evaluations appear in both rows. Top: validation loss versus $100 \cdot S_{\text{model}}$; vertical guides indicate A4/clipping-site and A7 analytic sparsity ceilings. Bottom: loss minus unmodified same-size A0 versus $100U_{\text{arch}}$, where $U_{\text{arch}} = S_{\text{model}} / S_{\text{model}}^{\text{max}}(\text{A7})$ for **every** curve. The A7 reference ceilings are 29.95%, 49.42% and 87.25%; the A4 guide is not the normalization denominator. At $\kappa = 0.5$, A7-OL1 improves both axes over A4-OL1 at all sizes; the ordering at smaller thresholds varies. Table 2 gives the boundary-threshold comparisons. Training-token budgets are matched across sizes.

Table 2: Complete-recipe contrasts at the boundary thresholds. Entries are A7-OL1 minus A4-OL1 at the same size and trained threshold. Negative loss changes and positive sparsity changes favor A7-OL1. All entries use complete validation. Table 7 reports all five thresholds.

Size	κ	Δ loss	ΔS_{model} (pp)
14M	0	+0.0219	-0.756
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.5	-0.0703	+9.024

4.5 Realizing sparsity with specialized inference kernels

Realizing activation sparsity requires kernels matched to the model’s shapes, nonlinearities and hardware. The released TwELL kernels (Cetin et al., 2026) accelerated their supported workload, but the unchanged small-width primitive failed numerical qualification for Pythia-14M. A shape- and sign-adapted descendant, P0, qualifies on only four of our 30 checkpoints, with geometric mean speedup $0.8910\times$ on that subset (Appendix D.4). This motivates specialization for the architecture and intervention.

To make this specialization practical within an architecture study, we use a human-guided coding-agent loop to propose, test and refine implementations. Execution and profiling feedback provide a basis for this approach (Ouyang et al., 2025; Luo et al., 2026). We replay 42 eligible proposals from the Codex search against native PyTorch scaled dot-product attention (SDPA) in CUDA graphs on one RTX5090. The workload is BF16, batch-one, uncached 2,048-token inference with full vocabulary logits. Qualification requires logits and loss to agree with the native reference within specified tolerances on complete validation.

The matched retrospective reaches $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42 on the fixed A7-OL1, $\kappa = 0.5$ search checkpoint (Figure 5, left): substantial gains appear early, followed by a later improvement. The selected implementation K050 qualifies on all 30 checkpoints in the 14M cohort, with geometric mean speedup $1.2340\times$. Speedup has a strong, approximately linear association with model-wide sparsity (right; OLS $R^2 = 0.8167$). This connects the logical metric to measured execution, while checkpoint quality, topology and weights vary across the fit.

Ablations distinguish fusion from the contribution of sparse paths. Relative to the same fusion with sparse paths disabled, K050 is $1.311\times$ as fast at the search checkpoint and $1.043\times$ as fast geometrically across the cohort, helping 14 of 30 checkpoints. Attention skipping is unprofitable in this workload: disabling it improves every checkpoint, raising the mean to $1.2506\times$. The attention kernel can skip a matrix-multiply instruction when either complete operand fragment is zero; matching zeros in both Q and K are unnecessary. Yet even substantial instruction skipping fails to repay its overhead at 14M (Appendix D.4). The full-model gain thus combines fusion with a conditional benefit from sparse paths: profitable execution depends on both sparsity and the granularity at which work is skipped.

Sparse kernel auto-research for full-model acceleration

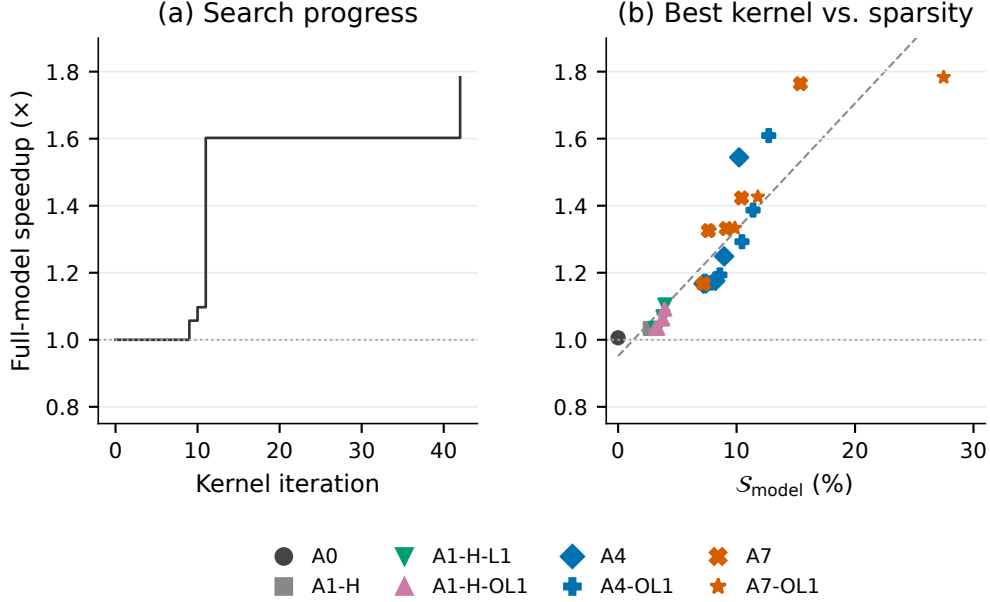


Figure 5: Sparse kernel auto-research for full-model acceleration. (a) Matched retrospective of the best qualified incumbent on fixed A7-OL1, $\kappa = 0.5$, across 42 iterations, starting at native $1\times$ and reaching $1.7830\times$. (b) The selected K050 implementation on 30 Pythia-14M checkpoints; symbols identify recipes. The dashed line is a descriptive OLS fit with intercept ($R^2 = 0.8167$); dotted lines mark native $1\times$. Speedup uses each checkpoint’s native SDPA CUDA-graph baseline on one RTX5090: BF16, batch one, uncached 2,048-token inference and full vocabulary logits. Qualification covers all 338 validation blocks. All checkpoints qualify, with geometric mean $1.2340\times$. Here “best” refers to the searched proposals; the attention-dense ablation is faster on every checkpoint (Table 10). Sparsity uses FP16 measurements, while timing uses BF16; both speedup axes have restricted ranges. The fit describes association across different checkpoints, not a causal sparsity effect.

5 Discussion and conclusion

Pressure, thresholds and site placement need to be evaluated together. In the matched 14M comparisons, adding pressure ranges from improving both loss and sparsity to exchanging quality for additional zeros. Extending the intervention to internal attention can increase model-wide sparsity even when FFN sparsity falls. The high-threshold seven-site recipe with pressure improves both axes over the four-site recipe at all three sizes, but the zero-threshold ordering changes. These results favor evaluating each addition against its matched recipe, rather than choosing interventions from activation sparsity alone.

The interpretation is supported by a connected pattern across experiments. Matched comparisons identify how pressure’s added effect changes with the threshold; distribution measurements reveal which activations change; and product counts explain their model-wide consequence. The high-threshold A7-OL1 advantage recurs at all three sizes, while the zero-threshold reversals identify where that relationship stops holding. Agreement across these comparisons makes the result more informative than a favorable checkpoint or an isolated sparsity percentage.

Product accounting and kernel timing answer different questions. Counting zero-operand products shows where an intervention affects computation; the sparsity ceiling explains how architecture changes that opportunity. The kernel study confirms that some of it can reduce full-model latency,

but also shows that skipping attention work can cost more than executing it. Fusion and sparse-path ablations are therefore necessary to attribute a measured speedup.

These conclusions concern MiniPile pretraining at a fixed token budget, with a lower learning rate at 410M. Larger models test complete recipes; the distribution diagnostics cover 14M and describe responses rather than isolating a causal spillover mechanism. The kernel results use one RTX5090, full-sequence inference, native SDPA and matched fusion controls. Transfer to cached decoding, larger models and other hardware remains untested.

References

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liquan Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41311–41330. PMLR, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A Intervention definitions

A.1 Thresholding nonlinearities

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 Activation pressure and OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the nonlinearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation micro-batches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov and Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B Sparsity accounting

B.1 Exact counters

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \epsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 Sparsity ceiling and normalization

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C Experimental setup details

C.1 Pythia sites and nonlinearities

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 3 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. The recipe and pressure assignments are defined in Section 4.

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

Table 3: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

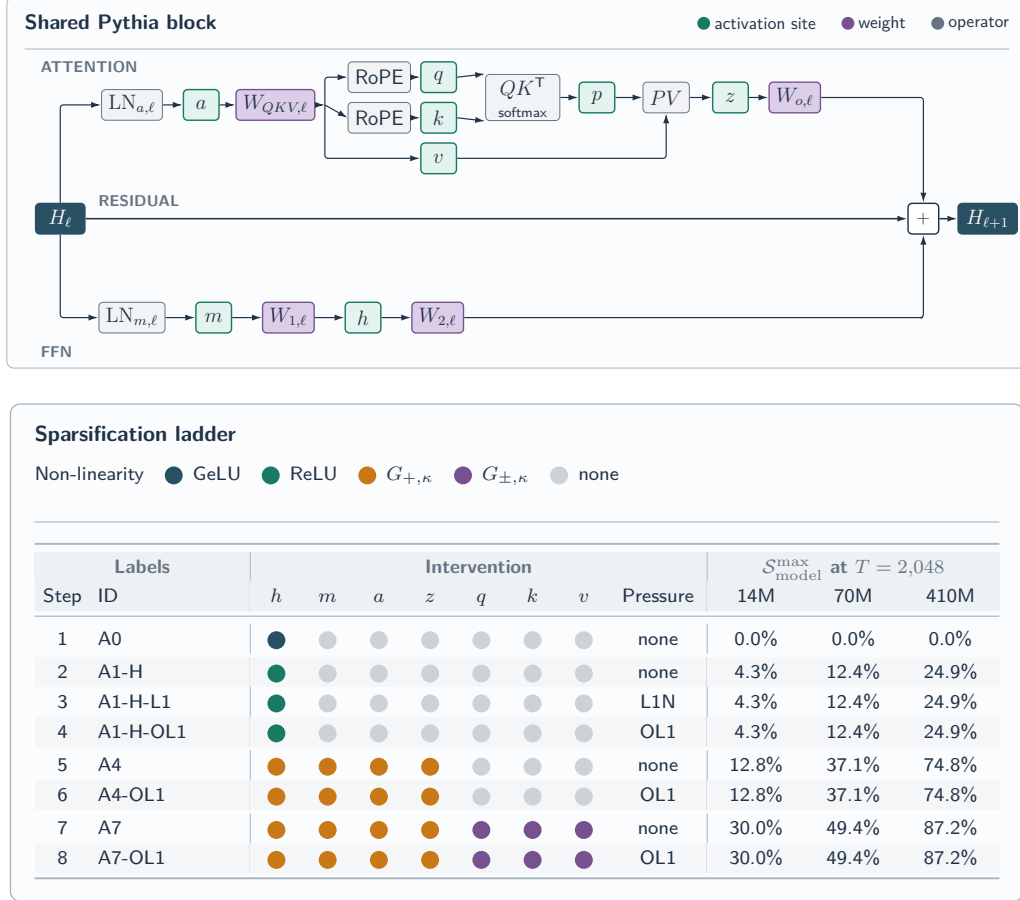


Figure 6: Shared Pythia block and sparsification intervention ladder. The map separates site placement, nonlinearity, and pressure; L1N denotes naive L1. The final columns show analytic selected-site reach $100 \cdot \mathcal{S}_{\text{model}}^{\text{max}} \%$ at $T = 2,048$, not observed sparsity or speedup. Pressure targets all selected sites in each pressure-bearing row. The ladder orders separately trained recipes, not curriculum stages; their experimental coverage is reported with the results.

C.2 Architecture-specific counts and sparsity ceilings

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV .

Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T + 1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \tag{4}$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

C.3 Evaluation scope

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared nonlinearities and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key–value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 Realized training protocol

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and training schedule. The 30-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds each for A4, A4-OL1, A7 and A7-OL1. Each larger cohort contains A0, A1-H, and the five-threshold A4-OL1/A7-OL1 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

at all four or seven selected sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\varepsilon = 10^{-12}$.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\varepsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 4 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 4: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

D Complete results and supporting diagnostics

The following tables report every selected trained endpoint, all 29 paired 14M contrasts and all 15 cross-scale recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 sparsity-ceiling denominator from Figure 4, rather than a different denominator for each recipe. The notation U_{arch} is shared with the main text.

D.1 Activation sparsity and signed-density coverage

The seven frozen-checkpoint evaluations underlying Figure 3 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 8 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional clipping. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences

Table 5: All 54 trained endpoints. λ is pressure weight; κ is the trained threshold. $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$ uses the same A7 reference within each model size, including for A0. Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	A4	$\kappa = 0$	5.4705	7.212	24.078
14M	A4	$\kappa = 0.01$	5.4665	7.414	24.752
14M	A4	$\kappa = 0.05$	5.4341	8.206	27.396
14M	A4	$\kappa = 0.1$	5.4197	8.953	29.891
14M	A4	$\kappa = 0.5$	5.6597	10.216	34.106
14M	A4-OL1	$\kappa = 0$	5.4583	7.810	26.075
14M	A4-OL1	$\kappa = 0.01$	5.4583	8.587	28.668
14M	A4-OL1	$\kappa = 0.05$	5.4898	10.456	34.910
14M	A4-OL1	$\kappa = 0.1$	5.5483	11.394	38.041
14M	A4-OL1	$\kappa = 0.5$	6.0380	12.713	42.446
14M	A7	$\kappa = 0$	5.4684	7.218	24.097
14M	A7	$\kappa = 0.01$	5.4588	7.618	25.433
14M	A7	$\kappa = 0.05$	5.4379	9.127	30.471
14M	A7	$\kappa = 0.1$	5.4287	10.425	34.805
14M	A7	$\kappa = 0.5$	5.7029	15.387	51.371
14M	A7-OL1	$\kappa = 0$	5.4802	7.054	23.551
14M	A7-OL1	$\kappa = 0.01$	5.4758	7.723	25.786
14M	A7-OL1	$\kappa = 0.05$	5.4628	9.863	32.929
14M	A7-OL1	$\kappa = 0.1$	5.4295	11.797	39.385
14M	A7-OL1	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	A4-OL1	$\kappa = 0$	4.8054	25.673	51.944
70M	A4-OL1	$\kappa = 0.01$	4.8229	26.224	53.060
70M	A4-OL1	$\kappa = 0.05$	4.8742	28.490	57.644
70M	A4-OL1	$\kappa = 0.1$	4.9439	30.574	61.861
70M	A4-OL1	$\kappa = 0.5$	5.3895	35.596	72.022
70M	A7-OL1	$\kappa = 0$	4.9412	23.562	47.674
70M	A7-OL1	$\kappa = 0.01$	4.9364	24.226	49.017
70M	A7-OL1	$\kappa = 0.05$	4.9637	26.524	53.666
70M	A7-OL1	$\kappa = 0.1$	4.9501	28.694	58.056
70M	A7-OL1	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	A4-OL1	$\kappa = 0$	5.6921	38.339	43.944
410M	A4-OL1	$\kappa = 0.01$	5.6914	39.149	44.873
410M	A4-OL1	$\kappa = 0.05$	5.6789	42.125	48.283
410M	A4-OL1	$\kappa = 0.1$	5.6789	45.688	52.367
410M	A4-OL1	$\kappa = 0.5$	5.1910	71.591	82.058
410M	A7-OL1	$\kappa = 0$	5.4263	41.121	47.133
410M	A7-OL1	$\kappa = 0.01$	5.4331	41.478	47.542
410M	A7-OL1	$\kappa = 0.05$	5.4679	44.053	50.494
410M	A7-OL1	$\kappa = 0.1$	5.4907	48.998	56.161
410M	A7-OL1	$\kappa = 0.5$	5.1207	80.616	92.401

Table 6: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1 – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1 – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1 – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1 – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1 – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1 – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1 – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1 – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1 – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1 – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 7: All matched A7-OL1 minus A4-OL1 comparisons. Negative loss changes and positive sparsity changes favor A7-OL1.

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 8: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1	0	FFN	67.22	0.0003	0.0000
A4-OL1	0	Attention	< 0.01	5.1717	0.6162
A4-OL1	0.05	FFN	86.71	0.0003	0.0000
A4-OL1	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1	0.5	FFN	99.31	0.0001	0.0000
A4-OL1	0.5	Attention	0.22	0.0003	0.0000
A7-OL1	0	FFN	64.52	0.0063	0.0000
A7-OL1	0	Attention	< 0.01	5.4657	0.4648
A7-OL1	0.05	FFN	75.29	0.0106	0.0000
A7-OL1	0.05	Attention	7.26	6.6719	0.6932
A7-OL1	0.5	FFN	93.64	0.0037	0.0000
A7-OL1	0.5	Attention	95.60	0.2396	0.0009

D.2 Operation contributions to model-wide sparsity

An operation’s contribution to $\mathcal{S}_{\text{model}}$ is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 9 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1 to A7-OL1 adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 7 displays the operation-level view at the same threshold and all three sizes.

Table 9: Contributions to $100\mathcal{S}_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1	A7-OL1	A4-OL1	A7-OL1	A4-OL1	A7-OL1
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

Operation contributions at $\kappa = .5$

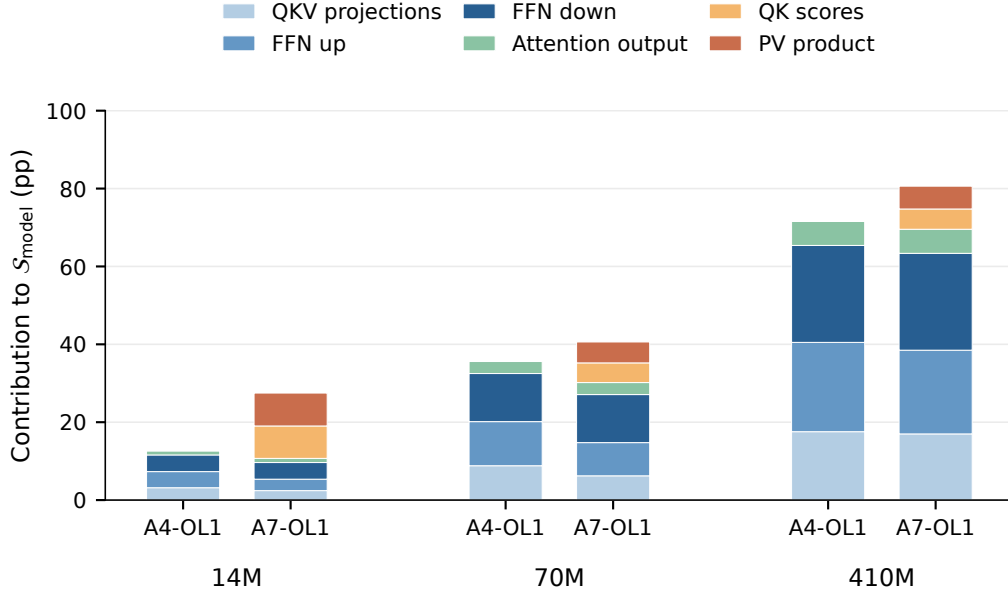


Figure 7: Operation accounting for A4-OL1 and A7-OL1 at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1, outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 Complete post-hoc clipping trajectories

The complete release contains 540 evaluations: ten clipping targets for each of the 54 trained checkpoints. It joins 350 new evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 8–10 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

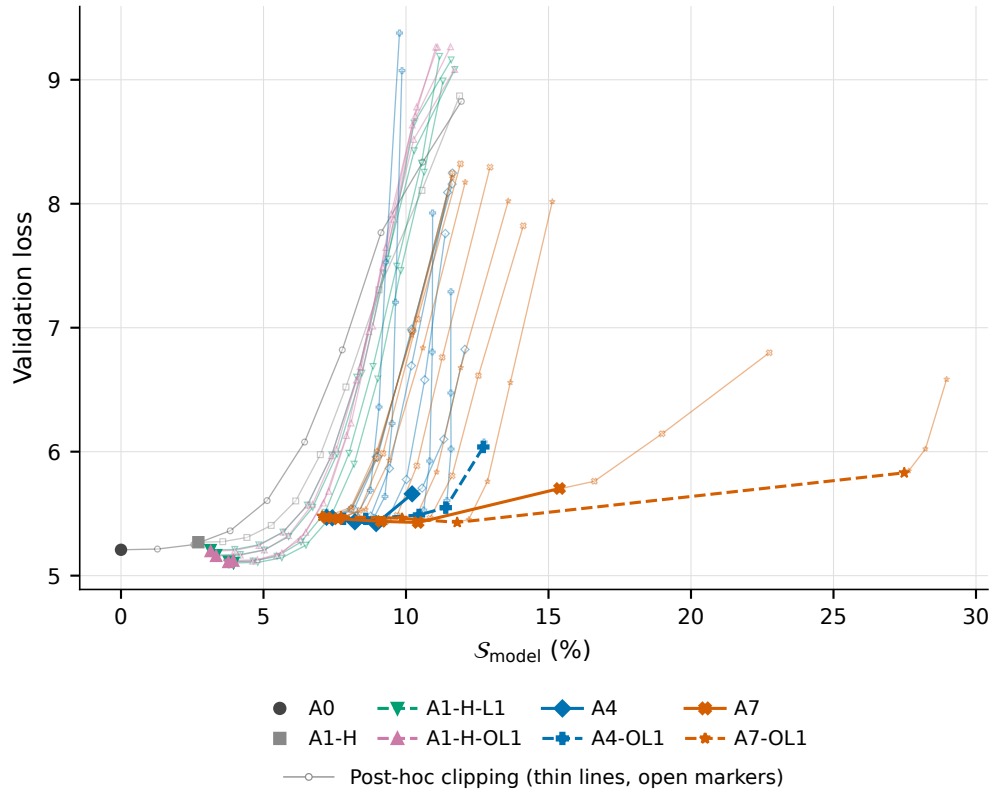


Figure 8: Complete 14M training and post-hoc clipping results: 30 trained endpoints and 300 clipping evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

70M training and post-hoc clipping

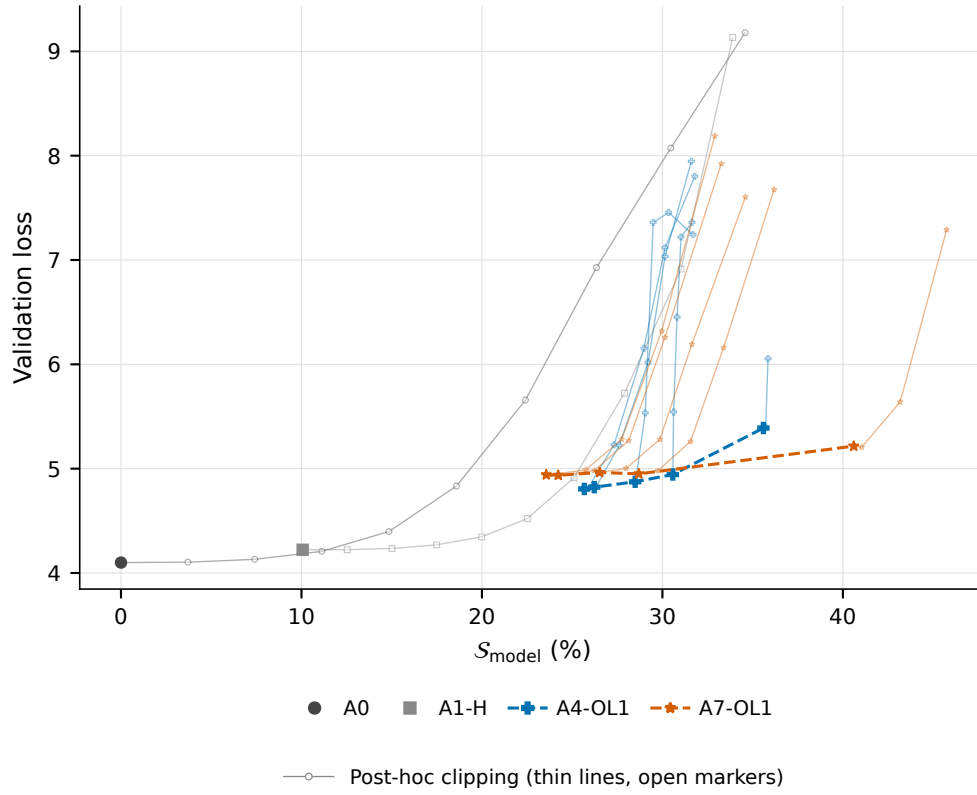


Figure 9: Complete 70M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

410M training and post-hoc clipping

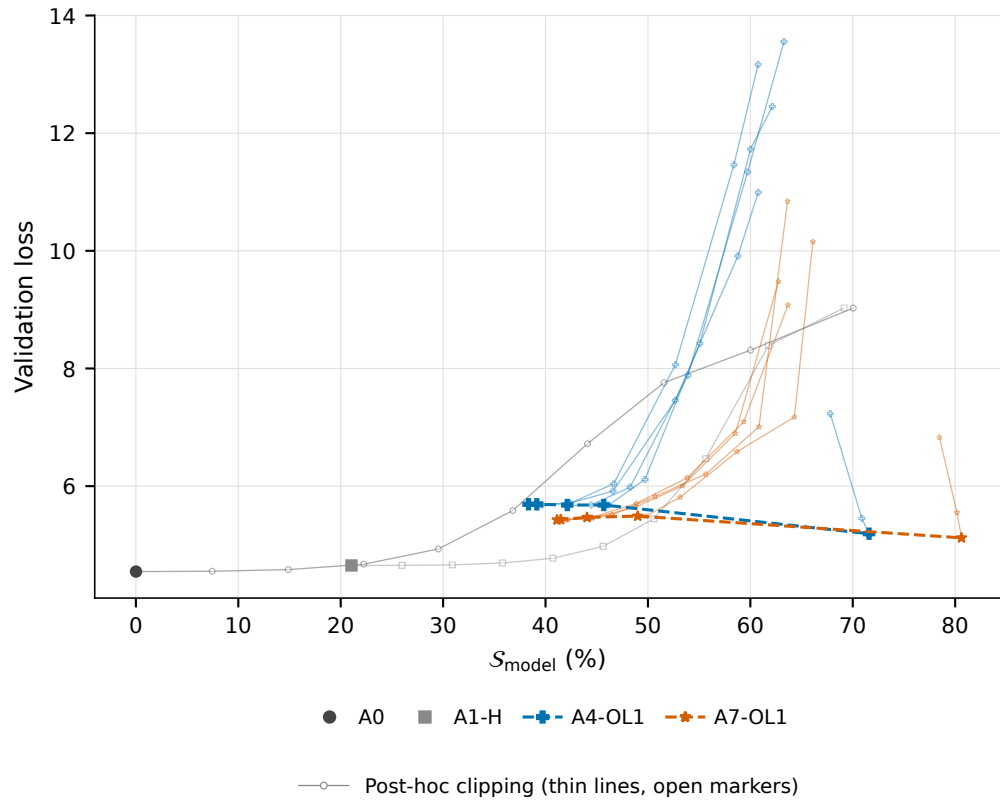


Figure 10: Complete 410M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

D.4 Kernel qualification and ablations

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 10 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the plotted incumbent uses fixed A7-OL1, $\kappa = 0.5$. Final implementation and ablation summaries retain the 30-checkpoint manuscript cohort from the historical 35-checkpoint sweep. Five A4 checkpoints with pressure only at h are excluded because they do not implement the four-site pressure objective studied here. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. Their ratios are formed from separately measured, native-normalized timings. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible *QK* and 67.2% of *PV* tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero *QK* score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Table 10: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 Evidence and data accompanying the draft

The supplementary directory `supplementary-data/` contains all 54 training records with integer operation counts, all 540 clipping evaluations and frontier memberships, seven signed histograms with zero and tail counts, figure coordinates, and the 30-checkpoint runtime reduction. The included README defines fields and units. Copy hashes are recorded in `SOURCES.json`; `protocol.json` gives data preparation, training settings and implementation identities. Paths refer to the accompanying repository; the supplementary directory is not a standalone checkpoint or kernel release.